# CIMR-V: An End-to-End SRAM-based CIM Accelerator with RISC-V for AI Edge Device

Yan-Cheng Guo[1] and, Tian-Sheuan Chang[1], *Senior Member, IEEE*,
Chih-Sheng Lin[2], Bo-Cheng Chiou[2], Chih-Ming Lai[2], Shyh-Shyuan Sheu[2], Wei-Chung Lo[2] and Shih-Chieh Chang[2]
[1]*Institute of Electronics, National Yang Ming Chiao Tung University,* Hsinchu, Taiwan
[2]*Industrial Technology Research Institute,* Hsinchu, Taiwan

2024 IEEE International Symposium on Circuits and Systems (ISCAS) | 979-8-3503-3099-1/24/$31.00 ©2024 IEEE | DOI: 10.1109/ISCAS58744.2024.10558177

***Abstract*—Computing-in-memory (CIM) is renowned in deep learning due to its high energy efficiency resulting from highly parallel computing with minimal data movement. However, current SRAM-based CIM designs suffer from long latency for loading weight or feature maps from DRAM for large AI models. Moreover, previous SRAM-based CIM architectures lack end-to-end model inference. To address these issues, this paper proposes CIMR-V, an end-to-end CIM accelerator with RISC-V that incorporates CIM layer fusion, convolution/max pooling pipeline, and weight fusion, resulting in an 85.14% reduction in latency for the keyword spotting model. Furthermore, the proposed CIM-type instructions facilitate end-to-end AI model inference and full stack flow, effectively synergizing the high energy efficiency of CIM and the high programmability of RISC-V. Implemented using TSMC 28nm technology, the proposed design achieves an energy efficiency of 3707.84 TOPS/W and 26.21 TOPS at 50 MHz.**

**Keywords : Computing-in-memory, AI accelerator, Pruning framework**

## I. Introduction

In recent years, convolution neural networks (CNNs) have had a profound impact on various applications, including but not limited to image classification [1] and speech processing [2], owing to their remarkable ability to extract features through very deep layers. However, the use of very deep layers in CNNs increases the memory bandwidth and requires larger computation demands, leading to massive data movement between memory and processing elements (PEs). This results in high latency and power consumption in current digital deep learning accelerators (DLAs). CIM offers an alternative approach by combining storage and computation, resulting in highly parallel computing and high energy efficiency [3].

However, previous SRAM-based CIM designs [4], [5] primarily focus on reducing the power consumed during weight data movement, while failing to address the need to minimize feature map data movement throughout the entire model execution. (Fig. 1) Moreover, as these deep learning models become deeper and larger to achieve higher accuracy, existing CIM designs with small array sizes will require frequent updates of model weights from DRAM, leading to high latency.

We propose CIMR-V to overcome the limitations mentioned above by reducing feature map data movement through CIM layer fusion and pipeline max pooling block, and reducing weight loading latency using the weight fusion method. Additionally, CIMR-V allows for user-friendly AI model deployment via the full stack flow. Implemented in TSMC 28nm technology, the proposed design achieves superior performance, with an energy efficiency of 3707.84 TOPS/W and a throughput of 26.21 TOPS at 50 MHz.

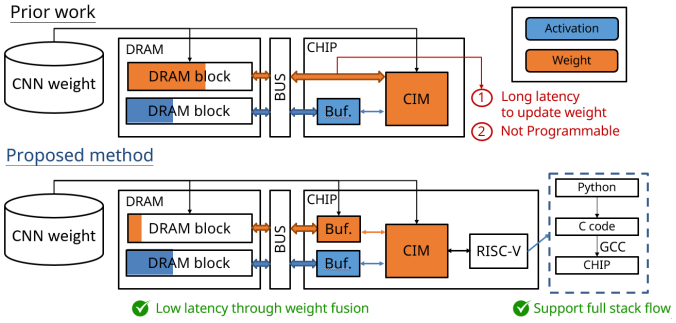



Fig. 1. The previous work failed to address the long latency involved in weight updates and lacked programmability. In contrast, our method incorporates a weight buffer to reduce latency and supports a full-stack flow integrating the RISC-V and CIM architecture.

The remainder of the paper is organized as follows. Section II shows the proposed hardware architecture and data flow. Section III presents the experimental results. Finally, this paper is concluded in Section IV.

## II. Proposed Hardware Architecture and Data Flow

### A. The Overall Architecture of CIMR-V.

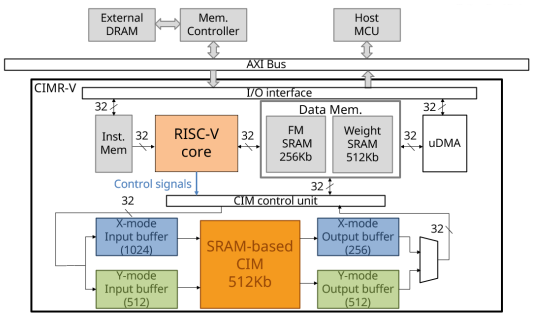



Fig. 2. The overall architecture of CIMR-V.

Authorized licensed use limited to: ShanghaiTech University. Downloaded on December 26,2024 at 08:43:20 UTC from IEEE Xplore. Restrictions apply.

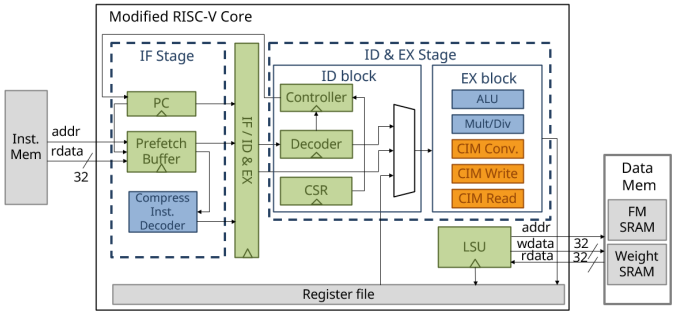
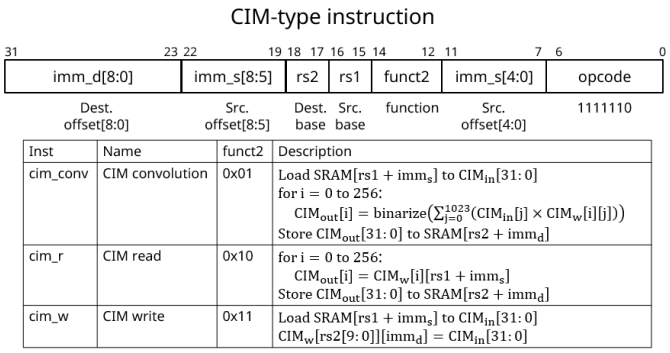



Fig. 3. The modified RISC-V core with CIM instruction for AI model inference.




| Inst | Name | funct2 | Description |
|---|---|---|---|
| cim_conv | CIM convolution | 0x01 | Load SRAM[rs1 + $imm_s$] to $CIM_{in}$[31:0]<br>for i = 0 to 256:<br>$CIM_{out}[i] = binarize(\sum_{j=0}^{1023}(CIM_{in}[j] \times CIM_w[i][j]))$<br>Store $CIM_{out}$[31:0] to SRAM[rs2 + $imm_d$] |
| cim_r | CIM read | 0x10 | for i = 0 to 256:<br>$CIM_{out}[i] = CIM_w[i][rs1 + imm_s]$<br>Store $CIM_{out}$[31:0] to SRAM[rs2 + $imm_d$] |
| cim_w | CIM write | 0x11 | Load SRAM[rs1 + $imm_s$] to $CIM_{in}$[31:0]<br>$CIM_w[rs2[9:0]][imm_d] = CIM_{in}[31:0]$ |

Fig. 4. The energy-efficient CIM-type instruction for CIMR-V.

Fig. 2 depicts the comprehensive architecture of CIMR-V, which is composed of a 512Kb SRAM-based CIM unit for energy-efficient computing, a modified RISC-V core for controlling the CIM and conducting high-precision computing, an instruction memory, a 256Kb feature map SRAM for layer fusion, and a 512Kb weight SRAM for weight fusion. CIMR-V is implemented on the PULPissimo platform, featuring the ibex 32-bit 2-stage RISC-V architecture [6]. In order to reduce routing complexity and power consumption, the CIM input buffer is designed with a 32-bit shift.

### *B. The High-density SRAM-based CIM Macro with X-mode and Y-mode*

This design is based on the SRAM-based CIM [7], which supports two distinct modes:

1) X-mode, which accommodates high input data with 1024 wordlines (WLs), 512 bitlines (BLs), and 256 sense amplifiers (SAs).
2) Y-mode, which accommodates high output data with 512 WLs, 1024 BLs, and 512 SAs.

To minimize power consumption and quantization error, the output of 512 / 1024 MAC operations is sensed by the SA on the corresponding long BL, and the activation function (ReLU) is executed simultaneously. Additionally, to enhance the robustness and accuracy of CNNs, we apply the symmetry weight mapping method to mitigate nonlinearity (NL) and cell variation in binary or ternary weights.

### *C. The Modified RISC-V Core with CIM Instruction*

Fig. 3 illustrates the modified RISC-V core, which incorporates not only the native Arithmetic Logic Unit (ALU) for basic arithmetic operations but also energy-efficient components, specifically the CIM read/write and CIM convolution modules.

To reduce pipeline overhead, we employ a two-stage CPU architecture. In the first stage, an instruction is fetched from memory using a prefetch buffer. In the second stage, the fetched instruction is decoded by the decoder unit, and the controller adjusts the program counter (PC) address and the control and status register (CSR) as needed. Simultaneously, the Load Store Unit (LSU) unit simultaneously loads data from data memory into the register file. Finally, in the execute block, the input data is computed and the output is stored in the register file.

To achieve optimal efficiency, all CIM instructions, such as the CIM convolution, write, and read instructions, are executed atomically within a single cycle, thereby minimizing latency and storage overhead. Additionally, to reduce feature map data movement, the CIM instructions utilize data from the feature map SRAM or weight SRAM instead of the register file.

### *D. The Energy-efficient CIM-type Instruction*

Fig. 4 illustrates the energy-efficient CIM-type instruction (opcode 1111110) employed in CIMR-V, which includes three main instructions: CIM convolution, CIM read, and CIM write. To improve system throughput, the CIM convolution instruction is executed within a single cycle by loading FM SRAM data from address register 1 (rs1) with the immediate source offset ($imm_s$) into the CIM input buffer, performing 512/1024 Multiply-Accumulate (MAC) operations, and storing the resulting CIM output into FM SRAM at rs2 with the added offset $imm_d$.

Similarly, the CIM read instruction reads the CIM macro weights from rs1 + $imm_s$ and stores them in SRAM at rs2 + $imm_d$. Additionally, the CIM write instruction supports the writing of 32-bit data in a single operation, further enhancing writing performance. Overall, this instruction set contributes to the energy efficiency and high performance of CIMR-V.

### *E. Row-wise Convolution Dataflow: CIM Layer Fusion and Pipeline with Convolution and Max Pooling*

The overall dataflow for the CIM convolution instruction is depicted in Fig. 5. This involves shifting the input feature map into the input buffer, flattening the CNN weights into macro BLs by output channel, and executing the convolution to obtain output data through SA.

To minimize power consumption and feature map data movement, the CIM convolution instruction supports CIM layer fusion, achieved by dividing the input feature map into blocks and storing overlapping data through the CIM input buffer to reduce layer fusion overhead. (Fig. 6) Furthermore, CIMR-V supports a pipeline with CIM convolution and max pooling to reduce latency and storage space. (Fig. 7)

Authorized licensed use limited to: ShanghaiTech University. Downloaded on December 26,2024 at 08:43:20 UTC from IEEE Xplore. Restrictions apply.

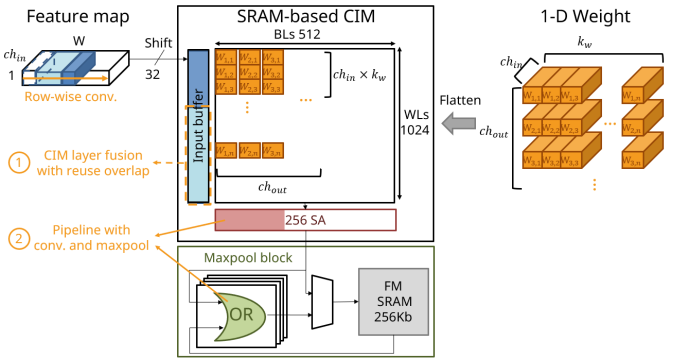



Fig. 5. The row-wise convolution dataflow enables CIM layer fusion [8] and pipeline with convolution and max pooling.

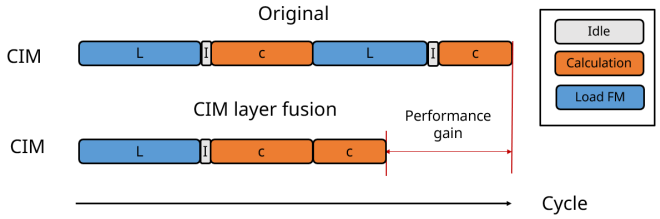



Fig. 6. The performance gain example of CIM layer fusion.

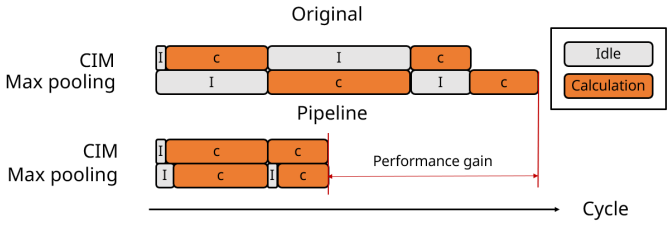



Fig. 7. The performance gain example of convolution/max-pooling pipeline.

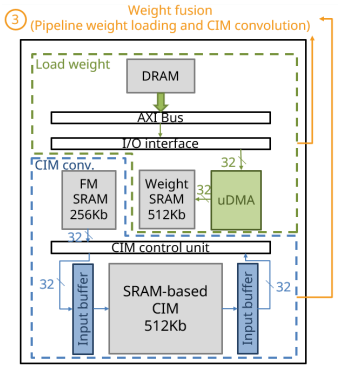



Fig. 8. The weight fusion pipeline combines weight loading and CIM convolution to minimize latency caused by weight loading from DRAM.

In general, row-wise convolution dataflow is crucial for enhancing throughput by utilizing CIM layer fusion and convolution/maxpool pipeline technology. Moreover, the dataflow is highly scalable to row-wise 2-D convolution.

### *F. Weight Fusion to Minimize Weight Loading Latency*

Fig. 8 depicts the proposed weight fusion method for CIM, which significantly reduces the weight loading latency from DRAM. This is achieved by pipelining the CIM convolution and weight loading into the weight SRAM, resulting in faster weight data access.

Furthermore, to minimize CPU loading, we employ uDAM for the parallel loading of weight data. uDAM is a technique that allows for direct data transfer between devices without the intervention of the CPU. This approach enables efficient weight data transfer and reduces CPU overhead.

Finally, the updated weight data from the weight SRAM is written into the CIM macro using the CIM write instruction. The proposed method offers a more efficient and streamlined weight loading process for CIM convolution, enhancing the overall performance of the system. (Fig. 9)

### *G. Full Stack Flow with CIMR-V for AI Models*

CIMR-V offers a full stack flow for model deployment, distinguishing it from conventional AI accelerators. The workflow involves converting the model's Python code, trained on TensorFlow or PyTorch, to C/C++ for deployment on CIMR-V using the GCC compiler and RISC-V architecture.

Furthermore, the full stack flow allows for seamless and efficient model deployment on CIMR-V, where CIM-type instructions effectively execute computation. This feature enables CIMR-V to deliver high performance with minimal latency, making it an excellent choice for AI workloads.

### *H. End-to-end AI Model Inference Flow of CIMR-V*

Fig. 10 presents the CIMR-V end-to-end flow for model inference through the RISC-V mode, the CIM mode, and the weight fusion mode. The pre-processing and post-processing steps that need high precision will be executed on RISC-V. The convolutional and pooling layers are executed through the CIM mode but with pipeline to reduce latency. Furthermore, the full model can be executed on chip without feature map I/O to DRAM through layer fusion processing. The corresponding weight update can be directly loaded from internal weight SRAM through weight fusion to avoid DRAM latency.

## III. Experimental Result

### *A. Network Simulation Result*

To showcase its efficacy, we adopt the keyword spotting task as an illustrative example. The architecture of the keyword spotting model, as illustrated in Fig. II, comprises preprocessing, 1-D binary convolution, max pooling, and post-processing. The model achieves an accuracy of 94.02% in the Google Speech Commands Dataset (GSCD) [10] with 12 classes.

Simulated latency optimization has included DRAM access latency based on DDR4 DRAM [11]. By means of layer fusion, the latency of the convolution execution can be effectively reduced by 33.16% due to the decreased movement of the feature map between the DRAM and the chip. Moreover,

Authorized licensed use limited to: ShanghaiTech University. Downloaded on December 26,2024 at 08:43:20 UTC from IEEE Xplore. Restrictions apply.

TABLE I
SUMMARY OF PERFORMANCE AND COMPARISON WITH OTHER DESIGNS.

| | JSSC'21 [4] | TCAS-1'22 [5] | ISSCC'22 [9] | **This work** |
|---|---|---|---|---|
| Technology | 65nm | 28nm | 22nm | **28nm** |
| Memory type | 6T SRAM | 6T SRAM | 6T SRAM | **10T SRAM** |
| Array size (WL×BL×sub-bank) | 128Kb (512×256×1) | 64Kb (16×64×16) | 576Kb (1152×512×1) | **512Kb (1024×512×1)** |
| IA(bits) | 4b/8b | 1b-8b | 2b/4b/8b(digital) / 7b(analog) | **1b** |
| W(bits) | 4b/8b | 1b-8b | 2b/4b/8b(digital) / 1.5b(analog) | **1b** |
| OA(bits) | - | 6b | - | **1b** |
| Supply voltage (V) | 1 | 0.8 | 0.55 | **0.9** |
| Frequency (MHz) | 1000 | 333.33 | 50-320 | **50** |
| Throughput (TOPS) | 0.0055 (0.352[1]) | - | 29.5 (309.75[1]) | **26.21 (26.21[1])** |
| Energy efficiency (TOPS/W) | 0.91 (166.91[2]) | 1280 (1011.36[2]) | 600 (1848.61[2]) | **3707.84 (3707.84[2])** |
| Algorithm | RAM | CNN | CNN | **CNN** |
| Dataset | GSCD | CIFAR100 | CIFAR10 | **GSCD** |
| Accuracy | 92.75% | 76.40% | 89.3%-91.4% | **94.02%** |
| End-to-end | - | - | ✓ | ✓ |
| Weight fusion | - | - | - | ✓ |

[1]Normalized operations = operations× activation precision× weight precision

[2]Normalized energy efficiency = energy efficiency× activation precision× weight precision× $\left(\frac{process}{28nm}\right)$ × $\left(\frac{voltage}{0.9V}\right)^2$

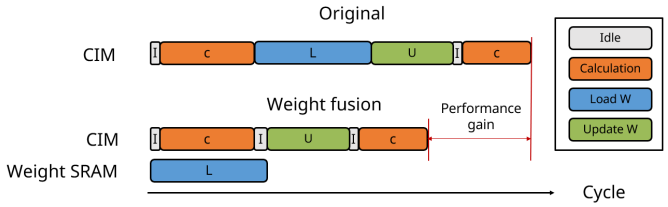



Fig. 9. The performance gain example of weight fusion.

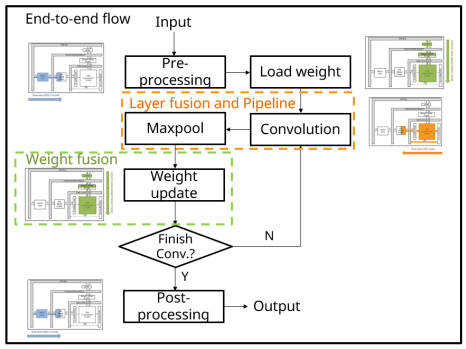



Fig. 10. CIMR-V end-to-end flow for AI model inference.

through weight fusion, an additional 62.94% of latency can be saved by minimizing weight transfer between the DRAM and the chip. Subsequently, by employing a pipeline for convolution and max pooling operations, another 40.00% of latency can be reduced by mitigating idle cycles of CIM macros. In summary, the proposed optimizations result in a significant 85.14% reduction in the end-to-end inference latency.

### *B. Implementation Results and Comparison*

The proposed digital design has been implemented with Verilog and synthesized with the Synopsys Design Compiler.

TABLE II
THE ARCHITECTURE OF OUR KEYWORD SPOTTING MODEL

| steps | input |
|---|---|
| preprocessing | high-pass filter, BN, quantize |
| convolution in CIM | (conv, max pooling) x5 |
| weight fusion | weight update |
| convolution in CIM | conv, max pooling, conv |
| post-processing | global average pooling |

Through the integration of the proposed design with the CIM macro, CIMR-V achieves 26.21 TOPS and 3707.84 TOPS/W with the TSMC 28nm CMOS process, when operating at a clock rate of 50 MHz.

Table I presents a summary of performance metrics and a comparative analysis with other SRAM-based CIM designs. In [4], a dedicated dataflow is proposed for specific algorithms, which cannot be reconfigured for alternative algorithms. Both [9] and [5] did not consider weight fusion, resulting in increased latency in transferring weights between DRAM and chip. Although [5] adopted multiple small CIM macros, which reduced energy efficiency due to overhead from peripheral circuits, the proposed design achieves higher energy efficiency and supports end-to-end model inference compared to other designs.

## IV. CONCLUSION

This paper proposes an end-to-end CIM accelerator with RISC-V, which supports CIM-type instructions and full stack flow. The entire design achieves an 85.14% reduction in execution latency for the keyword spotting model through the use of CIM layer fusion, weight fusion, and pipeline with convolution and maximum pooling. The proposed design, which was implemented using TSMC 28nm CMOS technology, attains an impressive 3707.84 TOPS/W and 26.21 TOPS with a 50 MHz clock rate.

Authorized licensed use limited to: ShanghaiTech University. Downloaded on December 26,2024 at 08:43:20 UTC from IEEE Xplore. Restrictions apply.

## REFERENCES


[1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," *Communications of the ACM*, vol. 60, no. 6, pp. 84–90, 2017.

[2] G. Chen, C. Parada, and G. Heigold, "Small-footprint keyword spotting using deep neural networks," in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2014, pp. 4087–4091.

[3] J.-M. Hung, C.-J. Jhang, P.-C. Wu, Y.-C. Chiu, and M.-F. Chang, "Challenges and trends of nonvolatile in-memory-computation circuits for AI edge devices," *IEEE Open Journal of the Solid-State Circuits Society*, vol. 1, pp. 171–183, 2021.

[4] H. Dbouk, S. K. Gonugondla, C. Sakr, and N. R. Shanbhag, "A 0.44-$\mu$J/dec, 39.9-$\mu$s/dec, recurrent attention in-memory processor for keyword spotting," *IEEE Journal of Solid-State Circuits*, vol. 56, pp. 2234–2244, 2021.

[5] Z. Yue, Y. Wang, Y. Qin, L. Liu, S. Wei, and S. Yin, "BR-CIM: An efficient binary representation computation-in-memory design," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 69, no. 10, pp. 3940–3953, 2022.

[6] P. D. Schiavone, D. Rossi, A. Pullini, A. Di Mauro, F. Conti, and L. Benini, "Quentin: an ultra-low-power PULPissimo SoC in 22nm FDX," in *IEEE SOI-3D-Subthreshold Microelectronics Technology Unified Conference (S3S)*, 2018, pp. 1–3.

[7] C.-S. Lin, F.-C. Tsai, J.-W. Su, S.-H. Li, T.-S. Chang, S.-S. Sheu, W.-C. Lo, S.-C. Chang, C.-I. Wu, and T.-H. Hou, "A 48 TOPS and 20943 TOPS/W 512kb computation-in-sram macro for highly reconfigurable ternary cnn acceleration," in *IEEE Asian Solid-State Circuits Conference (A-SSCC)*, 2021, pp. 1–3.

[8] M. Alwani, H. Chen, M. Ferdman, and P. Milder, "Fused-layer cnn accelerators," in *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2016, pp. 1–12.

[9] K. Ueyoshi, I. A. Papistas, P. Houshmand, G. M. Sarda, V. Jain, M. Shi, Q. Zheng, S. Giraldo, P. Vrancx, J. Doevenspeck, D. Bhattacharjee, S. Cosemans, A. Mallik, P. Debacker, D. Verkest, and M. Verhelst, "DIANA: An end-to-end energy-efficient digital and analog hybrid neural network SoC," in *IEEE International Solid- State Circuits Conference (ISSCC)*, vol. 65, 2022, pp. 1–3.

[10] P. Warden, "Speech commands: A dataset for limited-vocabulary speech recognition," 2018.

[11] Y. Kim, W. Yang, and O. Mutlu, "Ramulator: A fast and extensible dram simulator," *IEEE Computer Architecture Letters*, vol. 15, no. 1, pp. 45–49, 2016.


Authorized licensed use limited to: ShanghaiTech University. Downloaded on December 26,2024 at 08:43:20 UTC from IEEE Xplore. Restrictions apply.